our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Star Wolvez

SECURITY REVIEW

Date: 13 January 2026

CONTENTS

1. About Shieldify Security **3**

2. Disclaimer **3**

3. About Star Wolvez **3**

4. Risk classification **4**

4.1 Impact 5

4.2 Likelihood 5

5. Security Review Summary **5**

5.1 Protocol Summary 5

5.2 Scope 5

6. Findings Summary **5**

7. Findings **6**

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Star Wolvez

Overview

IB_Champions is an upgradeable ERC-721 NFT contract built with OpenZeppelin's UUPS (Universal Upgradeable Proxy Standard) pattern. It represents a collection of "Infinite Beyond Champions" NFTs with advanced features including dynamic pricing, token levelling, allowlist management, and role-based access control.

Key Features

1. Upgradeable Architecture

- Built using OpenZeppelin's UUPS proxy pattern
- Supports seamless contract upgrades while preserving state
- V2 upgrade available (`IB_ChampionsV2`) with additional features

2. ERC-721 NFT Implementation

- Standard ERC-721 token with URI storage
- Burnable tokens (when enabled by operator)
- Custom token names that can be changed by owners
- Token level system (all tokens start at level 1)

3. Minting System

Minting Modes:

- **Public Mint**: Anyone can mint by paying the required price
- **Allowlist Mint**: Only allowlisted addresses can mint (when enabled)
- **Role-based Mint**: `MINTER_ROLE` and `OPERATOR_ROLE` can mint without payment

Minting Controls:

- `publicMintEnabled`: Toggle public minting on/off (default: `true`)
- `allowlistEnabled`: Toggle allowlist requirement (default: `false`)
- Allowlist management via `updateAllowlist()` and `batchUpdateAllowlist()`

4. Dynamic Pricing System

The contract supports two pricing modes:

Fixed Pricing (Default)

- Set price in wei using `setMintPriceWei()`
- Default: 0.01 ETH

Dynamic Pricing (Pyth Oracle)

- Uses Pyth Network oracle for real-time ETH/USD price feeds
- Mint price set in USD cents via `mintPriceUSD` (e.g., 3000 = $30.00)
- Automatically calculates required ETH based on current ETH price
- Toggle via `toggleDynamicPricing(bool enable)`

Price Calculation Formula:

```
requiredETH = (mintPriceUSD * 10^8 * 10^18) / (ethPrice * 100)
```

5. Token Levelling System

Tokens can be upgraded to higher levels with operator signatures:

- All tokens start at **level 1**
- Levels can only increase (no downgrades)
- Requires a valid signature from `OPERATOR_ROLE`
- Protected against:
  - Replay attacks (nonce system)
  - Expired signatures (deadline check)
  - Cross-contract attacks (contract address in signature)
  - Cross-chain attacks (chain ID in signature)

Learn more about Star Wolvez's concept and the technicalities behind it here

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 2 days with a total of 128 hours dedicated to the audit by two researchers from the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying one Medium and three Low severity findings. They're mainly related to excess ETH amount not refunded to the user, validation gaps in supply reporting, oracle price usage, and wrong allowlist enforcement.

The Star Wolvez team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	**Star Wolvez**
Repository	sw_champions_contract
Type of Project	GameFi, ERC-721, Pyth Oracle
Security Review Timeline	2 days
Review Commit Hash	ce52b9bee1fd07b146df46828615c5c24c6e4d34
Fixes Review Commit Hash	c5f9dbb5fd82fe233c595dff75ea88a2e3afb3c2

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
src/IB_Champions.sol	269
Total	269

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: **1**
- **Low** issues: **3**
- **Info** issues: **5**

ID	Title	Severity	Status
[M-01]	Excess `msg.value` Is Not Refunded To the User	Medium	Fixed
[L-01]	`totalSupply` Returns Minted Count Instead of Circulating Supply	Low	Fixed
[L-02]	Pyth Oracle Confidence Interval Is Not Validated Before Price Usage	Low	Fixed
[L-03]	Allowlist Enforcement Applies Only to Mint Caller, Allowing NFTs to Be Minted or Transferred to Non-Allowlisted Addresses	Low	Fixed
[I-01]	`ownerOf` Existence Checks Never Hit Custom Errors	Info	Fixed
[I-02]	Hardcoded Pyth Exponent Can Misprice Dynamic Mint Costs	Info	Fixed
[I-03]	Mint Price Setters Should Enforce Constraints	Info	Fixed
[I-04]	Dynamic Pricing Lacks Slippage Protection, Exposing Users to Overpayment	Info	Fixed
[I-05]	Allowlist Restrictions Can Be Bypassed When Public Minting Is Enabled	Info	Fixed

7. Findings

[M-01] Excess `msg.value` Is Not Refunded To the User

Severity

Medium Risk

Description

The `safeMint()` function allows users to mint NFTs by paying ETH (the context). The function checks if the `msg.value` sent by the user is greater than or equal to the required amount, either calculated dynamically or using a fixed price (the context).

However, the function does not calculate or refund the excess ETH if the user sends more than the required amount (the problem).

This means that if a user accidentally sends more ETH than the mint price (e.g., due to a UI error or overestimation), the excess funds are retained by the contract and cannot be retrieved by the user, effectively resulting in a loss of funds for the user (the impact).

Location of Affected Code

File: contracts/IB_Champions.sol#L108-L137

```
function safeMint(address to, string memory uri, string memory _name)
   public payable {
    // code
    if (!hasRole(MINTER_ROLE, msg.sender) && !hasRole(OPERATOR_ROLE, msg.
       sender)) {
        if (useDynamicPricing) {
            uint256 requiredETH = getRequiredETHAmount();
            // @audit allows overpayment without refund
            require(msg.value >= requiredETH, "IB_Champions: Insufficient
                payment - below minimum USD value");
        } else {
            // @audit allows overpayment without refund
            require(msg.value >= mintPriceWei, "IB_Champions:
               Insufficient payment");
        }
    }
    // code
}
```

Impact

Users can lose ETH if they send more than the required mint price. The excess ETH is trapped in the contract until withdrawn by the admin, but there is no guarantee or mechanism for the admin to identify and return these overpayments to the specific users.

Proof of Concept

1. Assume `mintPriceWei` is set to 0.1 ETH.
2. A user calls `safeMint()` with `msg.value = 1 ether`.
3. The `require(msg.value >= mintPriceWei, ...)` check passes.
4. The contract mints the NFT.
5. The contract balance increases by 1 ETH.
6. The user has paid 1 ETH for an item costing 0.1 ETH, losing 0.9 ETH with no automatic refund.

Recommendation

Calculate the excess payment and refund it to the user. To follow the Checks-Effects-Interactions (CEI) pattern and prevent reentrancy attacks, ensure the refund is performed **after** all state changes and internal logic, preferably at the end of the function and use the `nonReentrant` modifier.

```
function safeMint(address to, string memory uri, string memory _name)
   public payable nonReentrant {
    // ... (Authorization checks) ...

    uint256 price = 0;
    if (!hasRole(MINTER_ROLE, msg.sender) && !hasRole(OPERATOR_ROLE, msg.
       sender)) {
        if (useDynamicPricing) {
            price = getRequiredETHAmount();
            require(msg.value >= price, "IB_Champions: Insufficient
               payment - below minimum USD value");
        } else {
            price = mintPriceWei;
            require(msg.value >= price, "IB_Champions: Insufficient
               payment");
        }
    }

    // Effects: Update state before external calls
    uint256 tokenId = _tokenIdCounter;
    _tokenIdCounter++;
    _tokenNames[tokenId] = _name;
    _tokenLevels[tokenId] = 1;

    // Interactions: External calls last
    _safeMint(to, tokenId);
    _setTokenURI(tokenId, uri);

    // Refund excess payment (CEI Pattern: Interaction at the end)
    if (msg.value > price) {
        (bool success, ) = payable(msg.sender).call{value: msg.value -
           price}("");
        require(success, "Transfer failed");
    }
}
```

Team Response

Fixed.

[L-01] TotalSupply Returns Minted Count Instead of Circulating Supply

Severity

Low Risk

Description

The contract exposes a `totalSupply()` view function that is likely to be used by UIs and integrators to determine collection supply metrics (the context). The contract also supports token burning

(when enabled by an operator), which can reduce the circulating supply over time (the context).

However, `totalSupply()` returns `_tokenIdCounter`, which is only the total number of tokens ever minted, and it is not reduced when tokens are burned (the problem).

This means integrators and users can be misled about the current circulating supply, potentially breaking supply-gated logic and analytics that assume `totalSupply()` reflects existing tokens (the impact).

Location of Affected Code

File: contracts/IB_Champions.sol

```
uint256 private _tokenIdCounter;

function totalSupply() external view returns (uint256) {
    return _tokenIdCounter;
}

function burn(uint256 tokenId) public override {
    // code
    super.burn(tokenId);
}
```

Impact

- `totalSupply()` reports **minted count**, not **current existing supply**.
- After burns, `totalSupply()` can be higher than the number of live tokens, causing UI/indexer discrepancies and supply-gating bugs in integrations.

Proof of Concept

1. Mint 1 token (tokenId 0). `_tokenIdCounter` becomes 1.
2. Enable burning and burn tokenId 0. The token no longer exists.
3. `totalSupply()` still returns 1, even though the circulating supply is 0.

Recommendation

- Either rename the function to `totalMinted()` (and document the meaning), or implement circulating supply tracking.
  - Example: maintain `uint256 public totalSupply;` that increments on mint and decrements on burn.
- Alternatively, use (or emulate) `ERC721Enumerable` semantics if that is the intended behavior.

Team Response

Fixed.

[L-02] Pyth Oracle Confidence Interval Is Not Validated Before Price Usage

Severity

Low Risk

Description

The contract integrates the Pyth ETH/USD price feed to support dynamic USD-based NFT pricing, but it does not validate the oracle's confidence interval (conf) before using the reported price. Pyth provides a confidence value alongside the price to indicate the level of uncertainty in the reported data, especially during volatile market conditions or temporary oracle degradation. By ignoring this confidence value and treating the price as exact, the contract may rely on low-quality or highly uncertain oracle data when calculating mint prices.

Location of Affected Code

File: contracts/IB_Champions.sol#L232-L243

```
function getETHPrice() public view returns (uint256) {
    require(address(pyth) != address(0), "IB_Champions: Pyth oracle not
        configured");

    PythStructs.Price memory price = pyth.getPriceNoOlderThan(
        ETH_USD_PRICE_ID, MAX_PRICE_AGE);
    require(price.price > 0, "IB_Champions: Invalid price data");

    // Pyth prices are in int64 format with expo, convert to uint256
    // For ETH/USD, expo is typically -8, so price is in 8 decimals
    uint256 ethPrice = uint256(uint64(price.price));

    return ethPrice;
}
```

Impact

If the oracle reports a price with a large confidence interval, users may be significantly overcharged or undercharged when minting NFTs. This can lead to incorrect pricing, unintended user fund loss, and unreliable mint behavior during periods of high volatility or oracle uncertainty.

Recommendation

The contract should validate the oracle confidence interval before accepting the reported price.

Team Response

Fixed.

[L-03] Allowlist Enforcement Applies Only to Mint Caller, Allowing NFTs to Be Minted or Transferred to Non-Allowlisted Addresses

Severity

Low Risk

Description

The allowlist enforcement in the `safeMint()` function only validates the caller (`msg.sender`) and does not impose any restriction on the recipient address (`to`). As a result, any address that is allowlisted can mint NFTs directly to arbitrary recipient addresses that are not themselves allowlisted. Furthermore, after minting, the NFTs behave as standard ERC-721 tokens and can be freely transferred to any address, regardless of allowlist status. If the intended behavior of the allowlist is to restrict not only who can initiate minting but also who can receive or hold NFTs during an allowlist phase, the current implementation does not enforce this constraint.

Location of Affected Code

File: contracts/IB_Champions.sol#L108-L137

```
function safeMint(address to, string memory uri, string memory _name)
    public payable {
    require(
        hasRole(MINTER_ROLE, msg.sender) ||
        hasRole(OPERATOR_ROLE, msg.sender) ||
        publicMintEnabled ||
        (allowlistEnabled && _allowlist[msg.sender]),
        "IB_Champions: Not authorized to mint"
    );
    require(bytes(_name).length > 0, "IB_Champions: Name cannot be empty"
        );
    require(bytes(_name).length <= 50, "IB_Champions: Name too long (max
        50 chars)");

    if (!hasRole(MINTER_ROLE, msg.sender) && !hasRole(OPERATOR_ROLE, msg.
        sender)) {
        if (useDynamicPricing) {
            uint256 requiredETH = getRequiredETHAmount();
            require(msg.value >= requiredETH, "IB_Champions: Insufficient
                payment - below minimum USD value");
        } else {
            require(msg.value >= mintPriceWei, "IB_Champions:
                Insufficient payment");
        }
    }
```

```
    // Effects: Update state before external calls
    uint256 tokenId = _tokenIdCounter;
    _tokenIdCounter++;
    _tokenNames[tokenId] = _name;
    _tokenLevels[tokenId] = 1; // All NFTs start at level 1

    // Interactions: External calls last
    _safeMint(to, tokenId);
    _setTokenURI(tokenId, uri);
}
```

Impact

During an allowlist-only minting phase, NFTs can still be distributed to non-allowlisted addresses through proxy minting by allowlisted users or subsequent transfers.

Recommendation

If the allowlist is intended to restrict NFT recipients and holders during the allowlist phase, the contract should enforce allowlist checks on the to address when allowlist minting is enabled, and optionally restrict transfers to non-allowlisted addresses during that phase.

Team Response

Fixed.

[I-01] OwnerOf Existence Checks Never Hit Custom Errors

Severity

Informational Risk

Description

Several view and admin functions attempt to validate token existence and provide a custom revert string (the context).

However, the checks are implemented using `ownerOf(tokenId) != address(0)`. In OpenZeppelin ERC-721, `ownerOf(tokenId)` reverts for non-existent tokens and never returns `address(0)`, so these checks are redundant and their custom revert strings are never emitted (the problem).

This means callers receive inconsistent revert reasons (from the underlying ERC-721 implementation) rather than the contract's intended error messages, and the extra `require` statements add unnecessary gas and complexity (the impact).

Location of Affected Code

File: contracts/IB_Champions.sol

```
function setTokenURI(uint256 tokenId, string memory uri) external
    onlyRole(OPERATOR_ROLE) {
     require(ownerOf(tokenId) != address(0), "IB_Champions: Token does not
         exist");
     _setTokenURI(tokenId, uri);
}

function getTokenName(uint256 tokenId) public view returns (string memory
    ) {
     require(ownerOf(tokenId) != address(0), "IB_Champions: Token does not
         exist");
     return _tokenNames[tokenId];
}

function getTokenLevel(uint256 tokenId) external view returns (uint256) {
     require(ownerOf(tokenId) != address(0), "IB_Champions: Token does not
         exist");
     return _tokenLevels[tokenId];
}
```

Impact

- Custom error messages are not actually reachable for non-existent tokens.
- Integrators expecting specific revert strings will observe ERC-721 reverts instead.
- Minor, but measurable gas waste from redundant checks.

Proof of Concept

Call `getTokenName(999)` (or `getTokenLevel(999)`) on a deployment where token 999 does not exist.

- Expected by the author: revert with `IB_Champions: Token does not exist`.
- Actual: revert from `ownerOf(999)` with the ERC-721 "nonexistent token" revert reason.

Recommendation

- Remove the redundant `require(ownerOf(tokenId) != address(0), ...)` and rely on the `ownerOf(tokenId)` revert.
- If you require custom errors/messages, use internal existence checks such as `_ownerOf(tokenId) != ad` (OZ v5) or `_requireOwned(tokenId)` and then revert with your own message.

Team Response

Fixed.

[I-02] Hardcoded Pyth Exponent Can Misprice Dynamic Mint Costs

Severity

Informational Risk

Description

The `safeMint()` function supports dynamic pricing by converting the USD-denominated mint price (`mintPriceUSD`, stored in cents) into Wei using the Pyth ETH/USD price feed (the context).

However, the conversion hardcodes an assumption that the Pyth price always uses 8 decimals (`PYTH_EXPO_DECIMALS = 1e8`) and does not use `PythStructs.Price.expo` returned by the oracle (the problem).

This means that if the feed exponent differs from the assumed value (oracle/feed upgrade, configuration mismatch, or non-standard oracle deployment), the contract can silently misprice mints, potentially undercharging users or requiring unexpected overpayment / causing reverts (the impact).

Location of Affected Code

File: contracts/IB_Champions.sol

```
uint256 private constant PYTH_EXPO_DECIMALS = 1e8; // Pyth price has 8
   decimals

function getETHPrice() public view returns (uint256) {
    // code
    PythStructs.Price memory price =
        pyth.getPriceNoOlderThan(ETH_USD_PRICE_ID, MAX_PRICE_AGE);
    require(price.price > 0, "IB_Champions: Invalid price data");

    // @audit ignores price.expo and assumes 8 decimals
    uint256 ethPrice = uint256(uint64(price.price));
    return ethPrice;
}

function getRequiredETHAmount() public view returns (uint256) {
    uint256 ethPrice = getETHPrice();

    // @audit assumes expo == -8 via PYTH_EXPO_DECIMALS
    uint256 requiredETH =
        (mintPriceUSD * PYTH_EXPO_DECIMALS * WEI_DECIMALS) /
        (ethPrice * USD_CENTS_DIVISOR);

    return requiredETH;
}
```

Impact

- If the Pyth exponent deviates from the assumed value, the mint price becomes incorrect.
- Users may be able to mint below the intended USD amount (undercharge).
- Users may be required to pay more than intended, or be unable to mint using UI-quoted amounts (overcharge / revert).

Proof of Concept

Assume:

- `mintPriceUSD = 3000` ($30.00)
- ETH/USD is $2000

If `expo = -8`, Pyth returns `price.price = 2000 * 10^8`, and the formula computes: required ETH = $30 / $2000 = 0.015 ETH (expected) If instead `expo = -10` for the same human-readable price, Pyth returns `price.price = 2000 * 10^10` (100x larger), but the contract still multiplies by `1e8`, so: required ETH becomes ~100x smaller (~0.00015 ETH), allowing mints far below the intended USD price

Recommendation

Do not hardcode the exponent scaling factor.

Safer options:

Incorporate `price.expo` returned by the Pyth Oracle into the conversion so the calculation remains correct if the exponent changes.

Team Response

Fixed.

[I-03] Mint Price Setters Should Enforce Constraints

Severity

Informational Risk

Description

The `safeMint()` function charges users either a USD-denominated mint price (dynamic pricing) or a fixed wei mint price (fixed pricing) (the context).

However, the `setMintPriceUSD()` and `setMintPriceWei()` operator setters accept any value without enforcing constraints such as non-zero minimums or sane upper bounds (the problem).

This means an operator mistake (or operator compromise) can unintentionally enable free mints (by setting the price to `0`) or effectively disable minting for regular users (by setting an extremely large price) (the impact).

Multiple functions should enforce 0 address checks and not allow the same value mutation.

Location of Affected Code

File: contracts/IB_Champions.sol

```
function safeMint(address to, string memory uri, string memory _name)
   public payable {
    // code
    if (!hasRole(MINTER_ROLE, msg.sender) && !hasRole(OPERATOR_ROLE, msg.
       sender)) {
        if (useDynamicPricing) {
            // @audit requiredETH depends on mintPriceUSD
            uint256 requiredETH = getRequiredETHAmount();
            require(msg.value >= requiredETH, "IB_Champions: Insufficient
                payment - below minimum USD value");
        } else {
            // @audit msg.value threshold depends on mintPriceWei
            require(msg.value >= mintPriceWei, "IB_Champions:
               Insufficient payment");
        }
    }
    // code
}

function setMintPriceUSD(uint256 newPriceUSD) external onlyRole(
   OPERATOR_ROLE) {
    // @audit no constraints (e.g., non-zero / max bounds)
    mintPriceUSD = newPriceUSD;
    emit MintPriceUpdated(newPriceUSD);
}

function setMintPriceWei(uint256 newPriceWei) external onlyRole(
   OPERATOR_ROLE) {
    // @audit no constraints (e.g., non-zero / max bounds)
    mintPriceWei = newPriceWei;
    emit MintPriceWeiUpdated(newPriceWei);
}
```

Impact

- Setting `mintPriceUSD = 0` (when `useDynamicPricing = true`) allows non-privileged callers to mint for free.
- Setting `mintPriceWei = 0` (when `useDynamicPricing = false`) allows non-privileged callers to mint for free.
- Setting either price to an extreme value can effectively disable minting for non-privileged callers.

Proof of Concept

1. Operator sets the dynamic mint price to zero:

```
setMintPriceUSD(0);
toggleDynamicPricing(true);
```

2. A public user can now mint with `msg.value = 0` while dynamic pricing is enabled:

```
safeMint(user, "ipfs://...", "name"); // msg.value = 0 succeeds
```

Similarly, under fixed pricing:

```
toggleDynamicPricing(false);
setMintPriceWei(0);
safeMint(user, "ipfs://...", "name"); // msg.value = 0 succeeds
```

Recommendation

Add explicit constraints to prevent accidental misconfiguration. For example:

- Enforce a non-zero minimum:
  - `require(newPriceUSD > 0, "...");`
  - `require(newPriceWei > 0, "...");`
- Consider enforcing a reasonable upper bound consistent with business requirements to reduce "disable minting by mistake" scenarios.
- If "free mint" is a desired operator feature, consider a dedicated boolean toggle for it instead of encoding it as `price = 0`.
- Also, enforce setters do not rewrite the same value, add checks that `fromValue != tovalue`, then only write value for all the required functions.

Team Response

Fixed.

[I-04] Dynamic Pricing Lacks Slippage Protection, Exposing Users to Overpayment

Severity

Informational Risk

Description

When dynamic pricing is enabled, the mint price in ETH is derived from the real-time ETH/USD price provided by the Pyth oracle at the moment of transaction execution. The safeMint function enforces only a minimum ETH requirement and does not provide any mechanism for users to specify a maximum acceptable ETH amount or slippage tolerance.

As a result, users may choose to send more ETH than required to avoid transaction reverts caused by price fluctuations, but if the ETH/USD price changes unfavorably between transaction submission and execution, users can end up paying more ETH than they initially intended without any on-chain protection.

Location of Affected Code

File: contracts/IB_Champions.sol#L108-L137

```
function safeMint(address to, string memory uri, string memory _name)
   public payable {
    require(
        hasRole(MINTER_ROLE, msg.sender) ||
        hasRole(OPERATOR_ROLE, msg.sender) ||
        publicMintEnabled ||
        (allowlistEnabled && _allowlist[msg.sender]),
        "IB_Champions: Not authorized to mint"
    );
    require(bytes(_name).length > 0, "IB_Champions: Name cannot be empty"
       );
    require(bytes(_name).length <= 50, "IB_Champions: Name too long (max
       50 chars)");

    if (!hasRole(MINTER_ROLE, msg.sender) && !hasRole(OPERATOR_ROLE, msg.
       sender)) {
        if (useDynamicPricing) {
            uint256 requiredETH = getRequiredETHAmount();
            require(msg.value >= requiredETH, "IB_Champions: Insufficient
                payment - below minimum USD value");
        } else {
            require(msg.value >= mintPriceWei, "IB_Champions:
               Insufficient payment");
        }
    }

    // Effects: Update state before external calls
    uint256 tokenId = _tokenIdCounter;
    _tokenIdCounter++;
    _tokenNames[tokenId] = _name;
    _tokenLevels[tokenId] = 1; // All NFTs start at level 1

    // Interactions: External calls last
    _safeMint(to, tokenId);
    _setTokenURI(tokenId, uri);
}
```

Impact

Users are exposed to unexpected overpayment during NFT minting when ETH price fluctuations occur between transaction submission and execution. This does not allow attackers to exploit the contract or cause direct protocol loss.

Recommendation

The minting function should support user-defined slippage control by allowing callers to specify a maximum acceptable ETH amount when dynamic pricing is used.

Team Response

Fixed.

[I-05] Allowlist Restrictions Can Be Bypassed When Public Minting Is Enabled

Severity

Informational Risk

Description

The contract does not enforce mutual exclusivity between allowlist-based minting and public minting. When the allowlist is enabled via `setAllowlistEnabled()`, public minting can remain enabled at the same time.

As a result, the allowlist mechanism does not strictly gate mint access, because the mint authorization logic permits minting whenever `publicMintEnabled` is true, regardless of the allowlist status. If the intended behavior is that enabling the allowlist should restrict minting exclusively to allowlisted addresses, this invariant is not enforced on-chain.

Location of Affected Code

File: contracts/IB_Champions.sol#L151-L154

```
function setAllowlistEnabled(bool enabled) external onlyRole(
    OPERATOR_ROLE) {
     allowlistEnabled = enabled;
     emit AllowlistStatusChanged(enabled);
}
```

Impact

If both `allowlistEnabled` and `publicMintEnabled` are set to true, any user can mint NFTs even when the allowlist is intended to be active. This can lead to unintended public access during restricted mint phases, breaking sale assumptions and potentially resulting in unfair NFT distribution.

Recommendation

When enabling the allowlist, the contract should automatically disable public minting to ensure that only allowlisted addresses are able to mint. This can be enforced by updating `setAllowlistEnabled()` to explicitly set `publicMintEnabled` to false whenever the allowlist is enabled, guaranteeing that allowlist-based minting cannot be bypassed due to misconfiguration.

Team Response

Fixed.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Thank you!